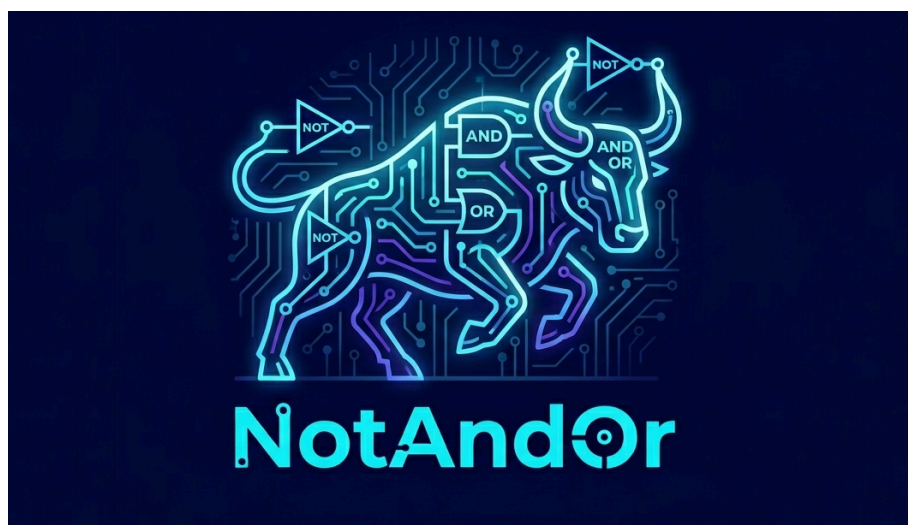




NotAndOr

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

8 de Marzo de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	4
6. Ejemplos	4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Manuel	Pandelo	65.748	mpandelo@itba.edu.ar
Santiago	Rapan	65.510	srapan@itba.edu.ar
Tomas	Costa Mendez	65.625	tcostamendez@itba.edu.ar

2. Repositorio

La solución será versionada en: [TPE-ATLC](#).

3. Dominio

El lenguaje propuesto tiene como objetivo la descripción y simulación de circuitos lógicos secuenciales síncronos. En este dominio, un sistema digital se compone de señales de entrada y salida, lógica combinacional, elementos con memoria y una señal de reloj que organiza la evolución temporal del circuito.

A diferencia de un circuito puramente combinacional, en un circuito secuencial la salida no depende únicamente de las entradas actuales, sino también del estado interno almacenado. Dicho estado se actualiza en instantes discretos, determinados por eventos del reloj. Por este motivo, el lenguaje debe permitir modelar tanto el comportamiento instantáneo de la lógica combinacional como la transición de estados entre ciclos.

El enfoque del lenguaje es funcional y estructural. Por un lado, permite expresar relaciones lógicas entre señales mediante operadores booleanos. Por otro, permite representar componentes reutilizables y componer circuitos de mayor complejidad a partir de subcircuitos. El resultado es una especificación formal del circuito, susceptible de ser analizada sintáctica y semánticamente, y posteriormente simulada.

El alcance del proyecto se limita al modelado de circuitos síncronos booleanos de tiempo discreto. En esta etapa no se consideran fenómenos físicos como retardos de propagación, rebotes, metastabilidad, señales analógicas ni múltiples dominios de clock. Tampoco se modelan buses o vectores multibit en la versión inicial del lenguaje. Todas las señales son booleanas y toman valores en el conjunto $\{0, 1\}$.

El dominio fue elegido porque ofrece:

- Un conjunto claro de abstracciones formales.
- Una distinción nítida entre lógica combinacional y secuencial.
- Múltiples restricciones semánticas interesantes para validar.
- Una implementación factible de un simulador sobre el compilador.
- Ejemplos y casos de prueba comprensibles y verificables.

4. Construcciones

El lenguaje permitirá describir circuitos mediante módulos llamados `circuit`, cada uno con una interfaz explícita de entradas y salidas, un conjunto de señales internas y una definición de su comportamiento.

4.1. Construcciones principales

Las construcciones previstas para la primera versión del lenguaje son las siguientes:

- Definición de circuitos:
 - Permite encapsular un componente lógico reutilizable.
 - Cada circuito tiene un nombre único y un cuerpo con declaraciones e instrucciones.
- Declaración de entradas:
 - Señales booleanas provistas desde el exterior del circuito.

- Se definen mediante la palabra clave input.
- Declaración de salidas:
 - Señales booleanas visibles desde el exterior del circuito.
 - Se definen mediante la palabra clave output.
- Declaración de cables internos:
 - Señales auxiliares sin memoria.
 - Se definen mediante la palabra clave wire.
- Declaración de registros:
 - Señales internas con estado persistente entre ciclos.
 - Se definen mediante la palabra clave reg.
- Asignaciones combinacionales:
 - Expresan relaciones lógicas instantáneas entre señales.
 - Utilizan el operador =.
- Bloques secuenciales:
 - Agrupan actualizaciones de estado que ocurren ante un evento de reloj.
 - Se expresan mediante una construcción del tipo `on rising_edge(clk) { ... }`.
- Asignaciones secuenciales:
 - Definen el próximo valor de un registro.
 - Utilizan el operador `<=`.
- Expresiones booleanas:
 - Compuestas mediante operadores como and, or, xor y not.
- Instanciación de subcircuitos:
 - Permite reutilizar circuitos definidos previamente y componer diseños más complejos.

4.2. Prestaciones del lenguaje

El lenguaje buscará ofrecer las siguientes prestaciones:

- Descripción declarativa de circuitos secuenciales.
- Separación explícita entre lógica combinacional y lógica de actualización de estado.
- Modularidad y reutilización de componentes.
- Simulación del comportamiento del circuito a lo largo de ciclos de reloj.
- Detección de errores estructurales y semánticos frecuentes en la descripción de hardware.

4.3. Restricciones semánticas previstas

Se prevé validar, entre otras, las siguientes condiciones:

- toda señal utilizada debe haber sido declarada;
- no puede haber redeclaración de identificadores en el mismo alcance;
- una señal de entrada no puede recibir asignaciones;
- una señal combinacional debe tener solo una definición.
- una asignación secuencial `<=` solo puede aparecer dentro de un bloque sensible al reloj;
- un reg solo puede actualizarse secuencialmente;
- una salida debe quedar determinada por alguna definición válida;
- una instancia debe referenciar un circuito existente;

- las conexiones de una instancia deben respetar la interfaz del circuito instanciado;
- no deben existir ciclos combinacionales ilegales.
- El identificador usado en `rising_edge` debe ser una señal declarada como input.

4.4. Modelo de ejecución previsto

La simulación del circuito se realizará en tiempo discreto, organizado en ciclos de reloj. En cada ciclo se considera el siguiente esquema conceptual:

1. Se leen los valores actuales de las entradas y de los registros;
2. Se evalúa la lógica combinacional;
3. Se calcula el próximo estado de los registros definidos en bloques secuenciales;
4. Al producirse el flanco ascendente del clock, los registros actualizan su valor.

De esta manera, el lenguaje permite representar de forma natural el comportamiento de circuitos secuenciales síncronos.

4.5 Sintaxis de instanciación

Una instancia de circuito tiene la forma:

```
CircuitName(
  input_port=signal,
  ...
) -> (
  output_port=signal,
  ...
);
```

Reglas:

1. Todos los puertos del circuito instanciado deben conectarse.
2. No se permiten puertos duplicados.
3. El orden de los puertos no es significativo.
4. Los nombres deben coincidir exactamente con la interfaz.
5. Las conexiones deben ser identificadores declarados.

5. Casos de prueba

A continuación se listan los casos de prueba iniciales previstos para la especificación del lenguaje.

5.1. Casos de aceptación

1. Definición de una compuerta combinacional simple con una entrada negada.
2. Definición de un HalfAdder con señales internas.
3. Definición de un DFlipFlop con un registro interno y salida asociada.

4. Definición de un TFlipFlop donde el estado se actualiza en función del estado previo.
5. Definición de un registro de un bit con señal de carga (load).
6. Composición de circuitos mediante instanciación de submódulos.
7. Circuito secuencial con varias señales internas y una única actualización de estado por ciclo.
8. Circuito con lógica combinacional que depende de entradas y de registros.
9. Simulación de un circuito durante múltiples ciclos de reloj.
10. Instanciación correcta de un subcircuito respetando todos sus puertos.

5.2. Casos de rechazo

1. Uso de una señal no declarada.
2. Redefinición de una señal dentro del mismo circuito.
3. Asignación a una señal de entrada.
4. Uso de `<=` fuera de un bloque `on rising_edge(...)`.
5. Múltiples definiciones incompatibles para una misma señal combinacional.
6. Instanciación de un subcircuito inexistente.
7. Instanciación de un subcircuito con puertos de entrada o salida inválidos respecto de su interfaz declarada.

6. Ejemplos

6.1. Ejemplo 1: compuerta combinacional simple

```

circuit InverterAnd {
  input a, b;
  output y;
  wire na;

  na = not a;
  y = na and b;
}
```

Código 6.1: el circuito calcula la negación de a y luego la combina con b mediante una conjunción lógica.

6.2. Ejemplo 2: half adder

```

circuit HalfAdder {
  input a, b;
  output sum, carry;
```

```
sum = a xor b;
carry = a and b;
}
```

Código 6.2: el circuito implementa un medio sumador de un bit. La salida sum representa la suma binaria sin acarreo previo, mientras que carry representa el acarreo generado.

6.3. Ejemplo 3: D flip-flop

```
circuit DFlipFlop {
  input d, clk;
  output q;
  reg state;

  q = state;

  on rising_edge(clk) {
    state <= d;
  }
}
```

Código 6.3: el circuito almacena en state el valor de d en cada flanco ascendente del clock. La salida q refleja el estado almacenado.

6.4. Ejemplo 4: T flip-flop

```
circuit TFlipFlop {
  input t, clk;
  output q;
  reg state;

  q = state;

  on rising_edge(clk) {
    state <= state xor t;
  }
}
```

Código 6.4: el circuito conmuta su estado cuando t = 1 y conserva el valor anterior cuando t = 0.

6.5. Ejemplo 5: registro de un bit con carga

```
circuit Register1 {
  input in, load, clk;
  output out;
  reg value;
```

```
wire next;

out = value;
next = (load and in) or ((not load) and value);

on rising_edge(clk) {
    value <= next;
}
}
```

Código 6.5: el circuito modela un registro de un bit. Si load vale 1, el siguiente estado toma el valor de in; en caso contrario, conserva su valor actual.

6.6. Ejemplo 6: instancia de circuito inexistente

```
circuit Main {
    input a, b;
    output y;

    UnknownGate(x=a, z=b) -> (out=y);
}
```

Código 6.6: el circuito UnknownGate no fue definido previamente, por lo que la instancia debe rechazarse semánticamente.

6.7. Ejemplo 7: puertos inválidos en una instancia

Suponiendo que exista el circuito:

```
circuit AndGate {
    input a, b;
    output out;

    out = a and b;
}
```

Entonces este uso debería rechazarse:

```
circuit Main {
    input x, y;
    output z;

    AndGate(left=x, right=y) -> (result=z);
}
```

Código 6.7: la instancia usa nombres de puertos left, right y result, pero el circuito AndGate define los puertos a, b y out. La conexión debe rechazarse por no respetar la interfaz declarada del subcircuito.